

Lab 02 — From the One-Time Pad to Stream Ciphers

IT4010E Introduction to Cryptography and Security · SolCT, HUST

Setup. Python 3.8+ and `pip install pynacl`. Write one program per task (task1.py, ..., task7.py). Print every key, pad and ciphertext **in hexadecimal** (`data.hex()`; for keys longer than 32 bytes, the first 32 bytes are enough). Hand in the programs and a report with their output and your explanations. Grading: tasks 1, 2, 5: 10 points each; tasks 3, 4, 6: 15 points each; task 7: 25 points (bonus +5). In each task, half of the points are for correct output and half for the explanations. On Windows there is no `/dev/urandom`: use WSL, or `os.urandom(size)`, which reads the same kind of source.

Messages used below:

```
M1 = "Send the final report to the dean before Friday noon."
M2 = "The exam for the course starts at eight in room D9."
M3 = "Lunch will be served in the main hall at half past twelve."
```

Task 1 — One-time pad

The lecture notes give this Python 2 code:

```
def random(size=16):
    return open("/dev/urandom").read(size)

def encrypt(key, msg):
    c = strxor(key, msg)
    print
    print c.encode('hex')
    return c

def main():
    key = random(1024)
    ciphertexts = [encrypt(key, msg) for msg in MSGS]
```

Port it to Python 3 and write `strxor` and `decrypt`. Generate a 1024-byte key from `/dev/urandom`, encrypt M1, M2 and M3, print the three ciphertexts in hex, decrypt them and print the plaintexts. Save the ciphertexts to `ciphertexts.txt`, one hex line per ciphertext.

Then encrypt a 1500-byte message with the same code and print the length of the result. Change `encrypt` so that it refuses a message longer than the key.

Explain: the error you get when running the original `random()` in Python 3, and what happened to the 1500-byte message.

Task 2 — Reusing the pad

Using only `ciphertexts.txt` (no key):

1. Compute and print $c_1 \oplus c_2$ in hex, and check that it equals $M_1 \oplus M_2$.
2. **Crib dragging:** slide the word "the " along $c_1 \oplus c_2$. At each position, XOR it in; if the result contains only lower-case letters and spaces, print the position and the result.
3. Find and print a key k' under which c_1 decrypts to "Nothing to see here." (padded with spaces to the length of c_1), and check it.

Explain: why the key disappears in step 1; why some positions in step 2 give real words and others do not (compare with M_1 and M_2); what step 3 means for an attacker who sees only c_1 .

Task 3 — A pseudorandom pad from NaCl

A 1 GB message needs a 1 GB one-time pad. Instead, we stretch a short random key into a long pad with a **pseudorandom generator** G , and encrypt $c = \text{nonce} \parallel (m \oplus G(k, \text{nonce}))$ with a fresh random 24-byte nonce per message.

PyNaCl's `SecretBox(key).encrypt(data, nonce)` XORs data with the XSalsa20 pseudorandom stream and puts a 16-byte tag in front of the result (`.ciphertext`). Encrypting n bytes equal to zero (`bytes(n)`) therefore gives the tag followed by n bytes of the pad itself.

Write $G(\text{key}, \text{nonce}, n)$ this way, plus `encrypt` and `decrypt`. Generate a 32-byte key and a 24-byte nonce with `nacl.utils.random`, and print them and $G(k, \text{nonce}, 64)$. Encrypt M_1 **twice**, print both ciphertexts, and decrypt one. Finally encrypt M_1 and M_2 with the **same** nonce (XOR each with the same $G(k, \text{nonce}, \cdot)$) and check whether $c_1 \oplus c_2 = M_1 \oplus M_2$.

Explain: why the two encryptions of M_1 differ, and what reusing a nonce does.

Task 4 — Encrypting a book

Download the textbook *A Graduate Course in Applied Cryptography* (Boneh and Shoup):

```
curl -O https://toc.cryptobook.us/book.pdf
```

Encrypt `book.pdf` into `book.enc` with **one** 32-byte key, reading the file 1 MiB at a time. Chunk number i uses the nonce prefix `|| i:` a random 16-byte prefix, written at the start of `book.enc`, followed by i on 8 bytes. Then decrypt `book.enc` into `book.dec.pdf` and open it in a PDF reader.

Print the key, the sizes of `book.pdf` and `book.enc`, the first 16 bytes of the plaintext and of the ciphertext (after the prefix), the SHA-256 of `book.pdf` and of `book.dec.pdf`, and the encryption time. (Tip: a byte-by-byte XOR in Python runs at about 25 MiB/s; XOR-ing the integers `int.from_bytes(a, "little")` and `int.from_bytes(b, "little")` is about ten times faster.)

Then encrypt the book again with the counter forgotten (every chunk uses prefix `|| 0`). For both versions, take the first two chunks c_0, c_1 of the ciphertext and m_0, m_1 of the book, and print whether $c_0 \oplus c_1 = m_0 \oplus m_1$.

Explain: the size of the key compared with the size of the book; why `book.enc` is exactly 16 bytes longer than `book.pdf`; what the forgotten-counter result shows, and why decrypting with the same buggy code would still give a book that opens correctly.

Task 5 — Changing an encrypted amount

Encrypt `"PAY BOB 0100 USD"` with your Task 3 cipher and print the ciphertext. **Without using the key**, change the ciphertext so that it decrypts to `"PAY BOB 9900 USD"`. Print the forged ciphertext and its decryption.

Repeat the same attack on a real `SecretBox` ciphertext (`box.encrypt(msg)`, layout: nonce 24 B, tag 16 B, body) and print what `box.decrypt` does.

Explain: why the first attack works, and what the 16-byte tag adds.

Task 6 — A shift-register generator

A **shift register** is a row of n bits; the seed is its initial content. Some fixed positions are called **taps** (position 0 is the rightmost bit). At each step: (1) output the rightmost bit, (2) compute the XOR of the bits at the taps, (3) shift the row one place to the right and insert that XOR on the left.

Row	Output	XOR of positions 0, 1	Next row
1001	1	1	1100
1100	0	0	0110
0110	0	1	1011

A seed is written as an integer whose bit i is the box at position i : seed 1001 is the integer 9, and “seed 1” means the row 00...01. Implement the register. Output bits are packed into bytes **bit 0 first**: the first output bit is bit 0 (value 1) of the first byte, the second is bit 1 (value 2), and so on. For example, the output bits 1 0 0 1 1 0 1 0 give the byte 0x59.

1. With $n = 4$, taps $\{0, 1\}$, seed 1001, print the first 16 output bits.
2. Print the number of steps until the row returns to the seed for: $n = 4$, taps $\{0, 1\}$, seed 1001; $n = 16$, taps $\{0, 2, 3, 5\}$, seed 1; $n = 16$, taps $\{0, 8\}$, seed 1.
3. With $n = 32$, taps $\{0, 10, 30, 31\}$ and a random seed, encrypt the message below ($\backslash n$ is a newline) and print the seed and ciphertext in hex. Print the fraction of 1-bits in 64 KiB of output.

```
"From: exam-office@example.edu\nSubject: final exam\nRoom B1-401, 8:00 on Monday."
```

Explain: why the seed must not be 0; why taps $\{0, 8\}$ would make a bad pad (think of Task 2).

Task 7 — Breaking the shift register

The ciphertext below was produced exactly as in Task 6.3 (same n , taps and bit order; unknown seed). The message starts with "From: ex".

```
3d5cae3331120fe78d2359b8998d846d5230dc78741f3880d0e36e3fb4659796
4a5841c015c4e29f25bbdc0f946993cd2af3984176325a9688ab8ab6d07dfda
f2f680eee88923ccc31738ce4c3c9962a5b92873b3064e64a69054def9c71331
```

Recover and print the seed (as an integer, in hex, e.g. `f"{seed:08x}"`) and the whole message (the last line is a flag).

Bonus: recover the message *without* using the taps, knowing only that the register has at most 32 bits.

Explain: why 8 known bytes are enough although the output of Task 6.3 looked random; whether a 256-bit register would be safe.